CONSTRUCCIÓN DE PÁGINAS WEB

HTML

CSS

JAVASCRIPT

@mardeasa

© Santiago Martín de Jesús

CREACIÓN DE PÁGINAS WEB CON EL LENGUAJE DE MARCAS

Formularios

@mardeasa

FORMULARIOS

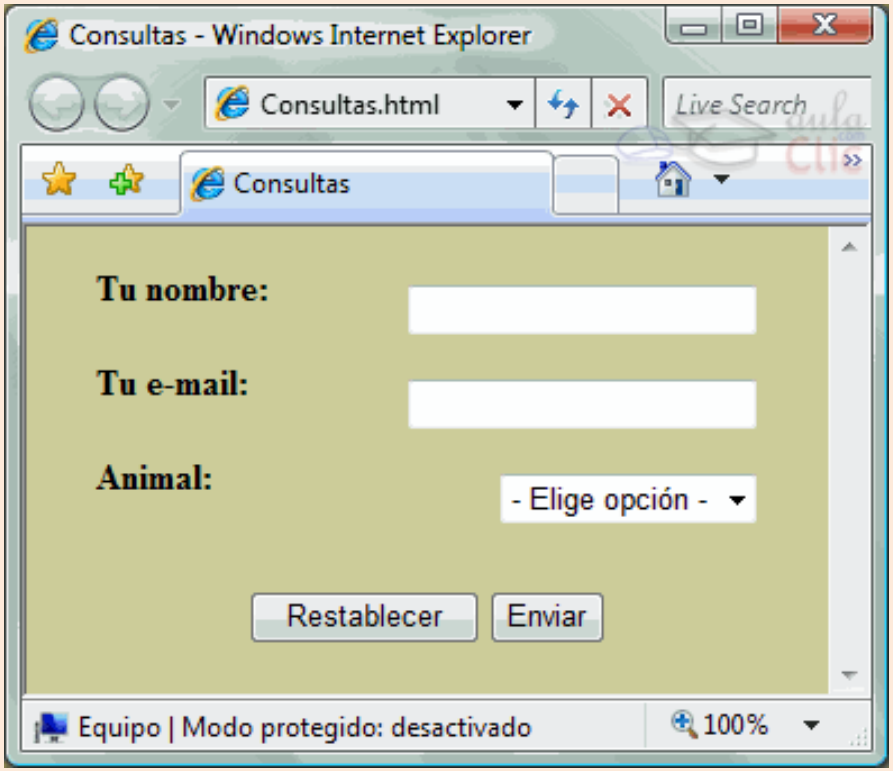
Los formularios se utilizan para recoger datos de los usuarios. Nos pueden servir para realizar un pedido en una tienda virtual, crear una encuesta, conocer las opiniones de los usuarios, recibir preguntas, etc.

Una vez el usuario rellena los datos y pulsa el botón para enviar el formulario se arrancará un programa que recibirá los datos y hará el tratamiento correspondiente.

Vamos a ver cómo crear el formulario, (insertar campos y botones en el formulario y validarlos), y la parte de tratamiento de los datos, para lo que se necesitan algunas nociones de programación, ya sea en PHP o en JavaScript.

FORMULARIOS

A la derecha tienes un ejemplo de formulario simple.

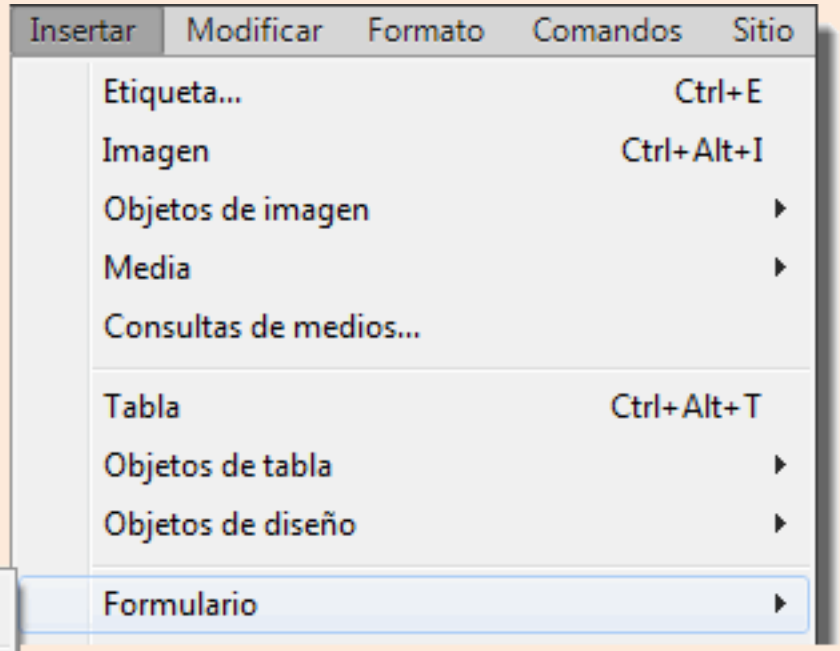


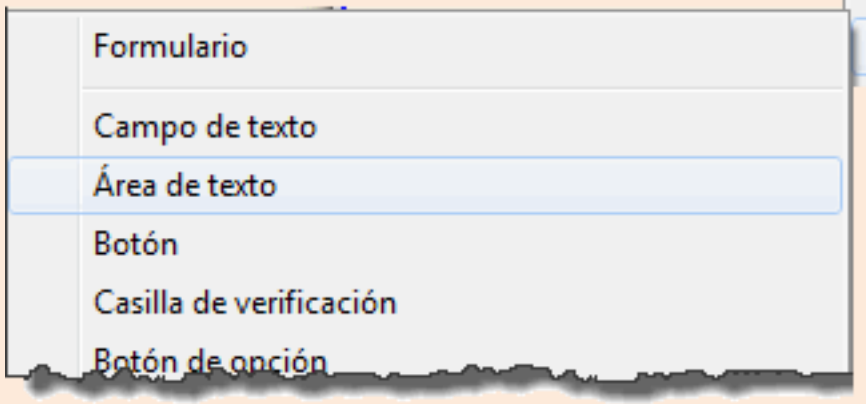
Un formulario está formado, entre otras cosas, por etiquetas, campos de texto, menús desplegables, y botones..

Una de las grandes novedades de HTML5 es la creación de nuevas etiquetas que nos asisten en la elaboración de estos formularios. HTML5 también incluye validaciones nativas para el navegador sin necesidad de usar javascript.

Elementos de formulario con Dreamweaver

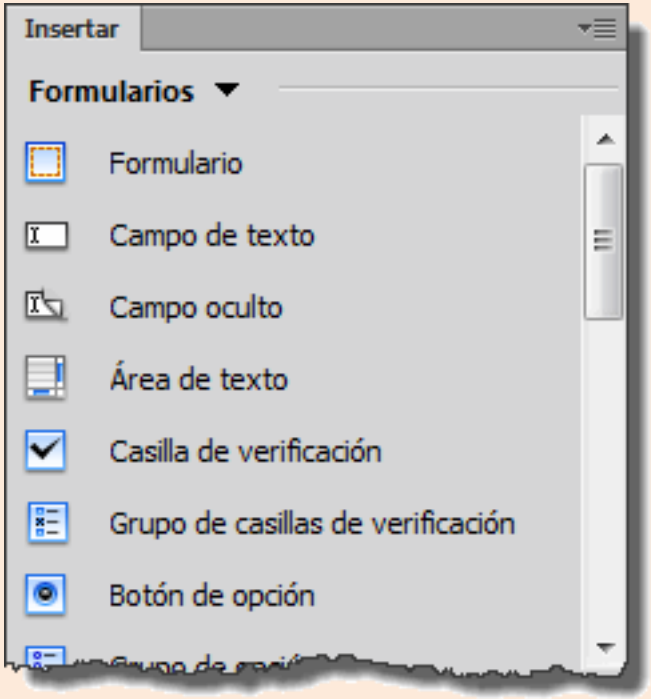
Los elementos de formulario pueden insertarse en una página desde Dreamweaver a través del menú Insertar, opción Formulario.





Elementos de formulario con Dreamweaver

o lo que es más cómodo si vamos a insertar varios elementos, desde el panel Insertar en la sección Formularios.



A través de esta opción se puede acceder a la lista de todos los objetos de formulario que pueden ser insertados en la página.

Ya veremos los distintos elementos que pueden formar parte de un formulario, así como algunas de sus propiedades. Antes de insertar estos elementos debemos crear el propio formulario tal y como veremos en el siguiente punto.

Crear formularios con Dreamweaver

Antes de insertar los elementos o controles del formulario, hemos de crear un formulario. Los elementos los introduciremos dentro de él, ya que cuando lo demos a enviar, sólo se enviarán los datos de los elementos de dentro del formulario.

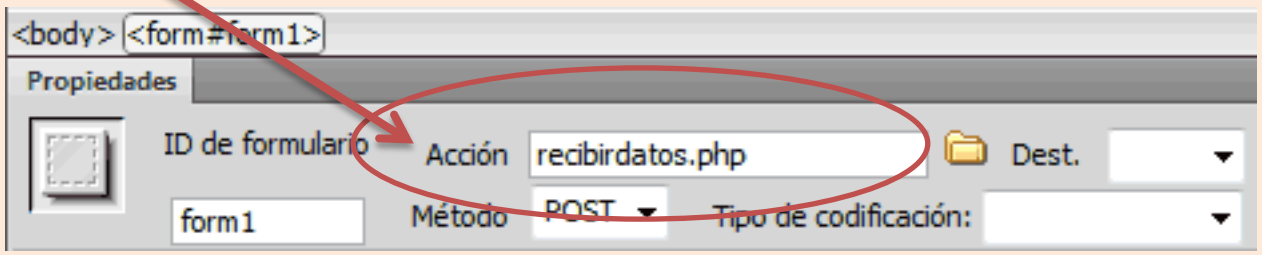
Puedes crear formularios a través del **menú Insertar → Formulario, opción Formulario**, o desde el panel Insertar.

Una vez creado un formulario, este aparecerá en la ventana de Dreamweaver como un recuadro formado por líneas naranjas discontinuas, similar al de la imagen siguiente.

Crear formularios con Dreamweaver

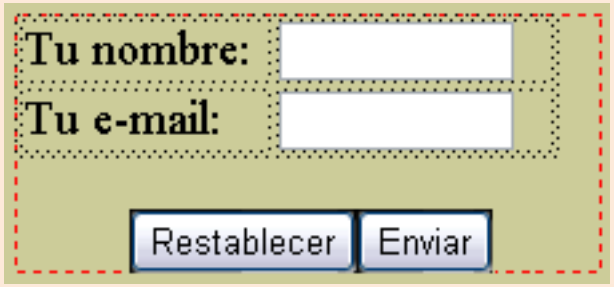
Estas líneas son ayudas visuales al trabajar en diseño, pero no se verán en el formulario final.

Entre las propiedades del formulario, encontramos el campo Acción. En el indicamos a dónde se envían los datos. Normalmente, una página PHP que se encargará de tratarlos.



Crear formularios con Dreamweaver

Dentro de dicho formulario se podrán insertar los elementos de formulario, que como ya sabes puedes insertar a través del menú Insertar, opción Formulario o desde el panel Insertar.



Es muy recomendable utilizar tablas para organizar los elementos de los formularios. Utilizando tablas se consigue una mejor distribución de los elementos del formulario, lo que facilita su comprensión y mejora su apariencia.

Podemos cambiar la apariencia de los formularios con CSS. En este caso, podemos emplear el selector form.

ETIQUETA FORM

Para crear formulario necesitamos usar la etiqueta FORM

<FORM> … </FORM>

La etiqueta <form> encierra todos los contenidos del formulario (botones, cuadros de texto, listas desplegables) y la etiqueta <input> permite definir varios tipos diferentes de elementos (botones y cuadros de texto).

Un formulario nos permite enviar datos al servidor, pero para procesar estos datos se utiliza un lenguaje de servidor (Recordar que HTML es lenguaje de cliente).

Atributos de la etiqueta FORM

- id = Representa el identificador o nombre del formulario para hacer referencia a éste desde CSS o scripts.

- action = "url" - Indica la URL que se encarga de procesar los datos del formulario.

- method = "POST o GET" - Especifica el método HTTP que se empleará para enviar los datos del usuario que ha introducido en el formulario. Los valores posibles son: get (por defecto) y post.

- enctype = "application/x-www-form-urlencoded o multipart/form-data" - Tipo de codificación empleada al enviar el formulario al servidor (sólo se indica de forma explícita en los formularios que permiten adjuntar archivos).

- accept = "tipo_de_contenido" - Lista separada por comas de todos los tipos de archivos aceptados por el servidor (sólo para los formularios que permiten adjuntar archivos).

Atributos de la etiqueta FORM

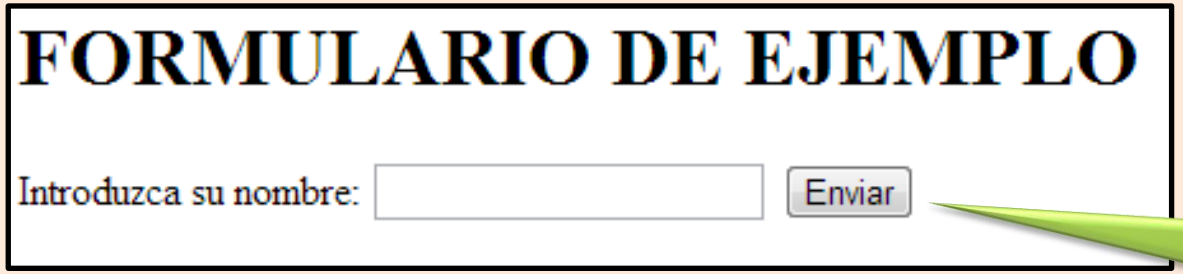
La mayoría de formularios utilizan sólo los atributos **id**, **action** y **method**. El atributo **action** indica la URL de la aplicación del servidor que se encarga de procesar los datos introducidos por los usuarios. Esta aplicación también se encarga de generar la respuesta que muestra el navegador.

El atributo **method** establece la forma en la que se envian los datos del formulario al servidor. Este atributo hace referencia al método HTTP, por lo que no es algo propio de HTML. Los dos valores que se utilizan en los formularios son **GET** y **POST**. De esta forma, casi todos los formularios incluyen el atributo method="get" o el atributo method="post".

Al margen de otras diferencias técnicas, el método ***POST*** permite el envío de mucha más información que el método GET. En general, el método GET admite como máximo el envío de unos 500 bytes de información. La otra gran limitación del método GET es que no permite el envío de archivos adjuntos con el formulario. Además, los datos enviados mediante GET se ven en la barra de direcciones del navegador (se añaden al final de la URL de la página), mientras que los datos enviados mediante POST no se pueden ver tan fácilmente.

Si no sabes que método elegir para un formulario, existe una regla general que dice que el método GET se debe utilizar en los formularios que no modifican la información (por ejemplo en un formulario de búsqueda). Por su parte, el método POST se debería utilizar cuando el formulario modifica la información original.

Ejemplo



¿Dónde se envían los datos?

```
<body>

<h1>FORMULARIO DE EJEMPLO</h1>

<form action="validar_formulario.php" method="post">
  Introduzca su nombre:
  <input type="text" name="nombre" value="" />
  <input type="submit" value="Enviar" />
</form>

</body>
```

Campos/controles que puede incluir <form>

```
<form ....>
        <input>
        <textarea>
        <button>
        <select>
        <option>
        <optgroup>
        <fieldset>
        <label>
        <datalist> nuevo en HTML 5
        <keygen> nuevo en HTML 5
        <output> nuevo en HTML 5
</form>
```

Elementos de formulario <input>

Los elementos de formulario como botones y cuadros de texto también se denominan "campos de formulario" y "controles de formulario". La mayoría de controles se crean con la etiqueta **<input>**, por lo que su definición formal y su lista de atributos es muy extensa:

- ✔ **type**: Indica el tipo de control de datos que se ha elegido. "*text, password, button, checkbox, reset, radio, hidden, file, image, submit* (html5) *color, date, datetime, datetime-local, email, month, number, range, search, tel, time, url, week.*"

Elementos de formulario <input>

- **src**: Para el control que permite crear botones con imagenes, indica el URL del archivo.
- **step**: (html5) permite establecer la cantidad de valores posibles dentro de un rango.
- ✔ **value**: indica el valor inicial del control.
- **autofocus**: (html5) fuerza el foco (la posición del cursor) dentro de un campo, una vez que se carga la página. En javascript conseguimos lo mismo con la función *focus()*.
- ✔ **required**: (html5) atributo que se aplica a un campo de entrada y obliga al usuario a escribir un valor en dicho campo. anteriormente se conseguía esto mediante javascript analizando la longitud del valor del campo en cuestión mediante *campo.value.length*.
- ✔ **placeholder**: (html5) muestra un texto por defecto en el input hasta que éste tiene el foco, en ese momento el texto desaparece para que el usuario introduzca el texto que desea. Anteriormente este efecto se creaba con javascript.

Elementos de formulario <input>

- ✔ **autocomplete**: (html5) activa *(on)* o desactiva *(off)* el auto completado de la entrada de datos en base a los textos introducidos anteriormente por el usuario. puede usarse como atributo de un campo de entrada o del formulario completo.
- **alt**: se utiliza cuando el tipo es una imagen y no se puede cargar en la interfaz del usuario, se muestra este texto alternativo.
- **checked**: para indicar si el elemento está seleccionado por defecto en los controles checkbox y radiobutton.
- **disabled**: es un valor booleano que indica que el elemento está desactivado, con lo cual no admite entrada de datos. Y su valor no se envía al Servidor junto al resto de datos.
- **readonly**: valor booleano que indica que el valor del campo no se puede modificar, solo lectura.
- **formenctype**: (html5) indica el modo de envío de los datos. aplication/x-www-form-urlencoded; multipart/form-data; text/plain
- **formaction**: (html5) es la dirección url que se utiliza al activar el elemento.

Elementos de formulario <input>

- **formnovalidate**: (html5) es un valor booleano que indica que el formulario no se valida antes de enviarse.
- **formtarget**: (html5) ventana del destino del formulario. *_blank, _self, _parent, _top, framename*
- **height**: (html5) altura del elemento tipo image.
- **width**: (html5) anchura del elemento tipo image.
- ✔ **size**: Tamaño inicial del control. En text/password indica la cantidad de caracteres que se pueden introducir en el campo, para el resto significa su tamaño en pixels.
- ✔ **max**: (html5) valor máximo que se puede entrar en el elemento de datos.
- ✔ **min**: (html5) valor mínimo que se puede entrar en el elemento de datos.
- ✔ **maxlength**:longitud máxima de caracteres para los campos text/password.
- **list**: (html5) es un identificador de una lista que se define con datalist.

Elementos de formulario <input>

- **multiple**: (html5) valor booleano que indica que se permite la selección de valores múltiples.
- ✔ **name**: Nombre que identifica al campo de datos. Importante para su identificación por el Servidor.
- ✔ **pattern**: (html5) Expresión regular que se utiliza para validar entradas de datos. este atributo permite hacer validaciones muy complejas y utilizado correctamente puede ahorrarse muchas lineas de código.

Elementos de formulario <input>

Atributo **<input type="valor">** Podemos incluir los siguientes valores:

Value	Descripción
button	Define un botón que se pulse (en su mayoría se utiliza con un JavaScript para activar una secuencia de comandos)
checkbox	Define una casilla de verificación
color New	Define un selector de color
date New	Define un control de fecha (año, mes y día (no hay tiempo))
datetime New	Define un control de fecha y hora (año, mes, día, hora, minuto, segundo y fracción de segundo, basado en la zona horaria UTC)
datetime-local New	Define un control de fecha y hora (año, mes, día, hora, minuto, segundo y fracciones de segundo (sin zona horaria)
email New	Define un campo para una dirección de e-mail
file	Define un campo de archivo de selección y un "Examinar ..." botón (para la carga de archivos)
hidden	Define un campo de entrada oculto
image	Define una imagen como botón submit
month New	Define un mes y control del año (sin zona horaria)
number New	Define un campo para introducir un número
password	Define un campo de contraseña (caracteres se enmascaran)
radio	Define un botón de opción
range New	Define un control para introducir un número cuyo valor exacto no es importante (como un control deslizante)
reset	Define un botón de reinicio (restablece todos los valores del formulario a valores por defecto)

Elementos de formulario <input>

Atributo **<input type="valor">**

Value	Descripción
search New	Define un campo de texto para introducir una cadena de búsqueda
submit	Define un botón de envío
tel New	Define un campo para introducir un número de teléfono
text	Default. Define un campo de texto de una sola línea (ancho predeterminado es de 20 caracteres)
time New	Define un control para entrar en un tiempo (sin zona horaria)
url New	Define un campo para introducir una URL
week New	Define a la semana y el control al año (no zona horaria)

Ejemplo

Aquí podemos descargar un ejemplo con todos los tipos de campos posibles en formularios.

Etiqueta <label>

Afortunadamente, el lenguaje HTML incluye una etiqueta denominada <label> y que se utiliza para establecer el título de cada campo del formulario.

Utilizando la etiqueta <label>, cada campo de formulario puede disponer de su propio título.

Ejemplo

```
<label for="nombre">Nombre</label> <br/>
<input type="text" id="nombre" name="nombre" value="" />

<label for="apellidos">Apellidos</label> <br/>
<input type="text" id="apellidos" name="apellidos" value="" />

<label for="dni">DNI</label> <br/>
<input type="text" id="dni" name="dni" value="" size="10" maxlength="9" />
```

La principal ventaja de utilizar <label> es que el código HTML está mejor estructurado y se mejora su accesibilidad. Además, al pinchar sobre el texto del <label>, el puntero del ratón se posiciona automáticamente para poder escribir sobre el campo de formulario asociado. Este comportamiento es especialmente útil para los campos de tipo radiobutton y checkbox.

Cajas de texto:
<input type="text" ...>

Visualiza una caja de texto del tamaño especificado por el atributo size.

Se emplea para pedir datos de entrada relativamente cortos de una línea como por ejemplo, nombre, apellidos, dirección, etc.

Atributos

- name: Asigna un identificador a la variable donde se guardara el valor del usuario también sirve como referencia en scripts aunque más recomendado usar id.
- value: Visualiza un valor predeterminado en el campo de entrada.
- size: Indica la longitud visible en número de caracteres que tendrá la caja de texto.
- maxlength: Indica la longitud máxima de caracteres que puede introducir el usuario.

Ejemplo

```
Nombre
<input type="text" name="nombre" value="Introduce tu nombre" size="30" maxlength="40"/>
```

Nombre Introduce tu nombre

Cajas de texto tipo contraseña:
<input type="password" ...>

Es muy similar al tipo text, con la diferencia de que el texto introducido en la caja aparece oculto, mostrando un símbolo * (asterisco) por cada carácter en el navegador. Como su nombre indica, sirve para introducir contraseñas y que nadie de nuestro alrededor pueda ver lo que se escribe.

Ejemplo

```
Introduce Contraseña <input type="password" name="pwd" />
```

Introduce Contraseña ••••••••••

NOTA: Las cajas de password solo impiden que se muestre el texto en pantalla, el texto no es codificado cuando se envía al servidor.

Campo <textarea > </textarea>

Genera un campo de texto multilínea en el que introducir textos largos, como descripciones, etc.

Consta de una etiqueta de apertura y de una de cierre, siendo obligatorio el uso de ambas. Entre estas solo puede insertarse texto llano, sin etiquetas HTML.

Consiste en una caja de filas y columnas dentro de la cual se introduce el texto.

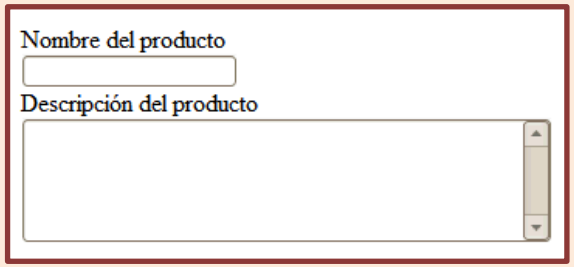
No dispone de límite en el número de caracteres, por lo que cuando el usuario introduce más texto del visualizado aparecen las barras de desplazamiento de la caja (scroll).

Atributos:

cols: Especifica la anchura o columnas visibles en número de caracteres.

rows: Especifica la altura o filas de texto visibles.

Ejemplo




```
<label for="nombre">Nombre del producto</label> <br/>
<input type="text" id="nombre" name="nombre" value="" />

<label for="descripcion">Descripción del producto</label> <br/>
<textarea id="descripcion" name="descripcion" cols="40" rows="5"></textarea>
```

Casillas de verificación:
<input type="checkbox" ...>

Los checkbox o "casillas de verificación" son controles de formulario que permiten al usuario seleccionar y deseleccionar opciones individualmente. Aunque en ocasiones se muestran varios checkbox juntos, cada uno de ellos es completamente independiente del resto. Por este motivo, se utilizan cuando el usuario puede activar y desactivar varias opciones relacionadas pero no excluyentes.

El valor del atributo type para estos controles de formulario es checkbox. Como se muestra en el ejemplo anterior, el texto que se encuentra al lado de cada checkbox no se puede establecer mediante ningún atributo, por lo que es necesario añadirlo manualmente fuera del control del formulario. Si no se añade un texto al lado de la etiqueta <input /> del checkbox, el usuario sólo ve un pequeño cuadrado sin ninguna información relativa a la finalidad de ese checkbox.

El valor del atributo value, junto con el valor del atributo name, es la información que llega al servidor cuando el usuario envía el formulario.

Si se quiere mostrar un checkbox seleccionado por defecto, se utiliza el atributo **checked**. En cualquier otro caso, el checkbox permanece sin seleccionar. Aunque resulta redundante que el nombre y el valor del atributo sean idénticos, es obligatorio indicarlo de esta forma porque los atributos en XHTML no pueden tener valores vacíos:

Ejemplo

```
Deportes practicados:
<input type="checkbox" value="futbol" id="futbol" name="deportes" checked>
<label for="futbol">Fútbol </label>
<input type="checkbox" value="baloncesto" id="baloncesto" name="deportes">
<label for="baloncesto">Baloncesto </label>
<input type="checkbox" value="ciclismo" id="ciclismo" name="deportes">
<label for="ciclismo">Ciclismo </label>
```

Deportes practicados: ☑ Futbol ☐ Baloncesto ☐ Ciclismo

Control radiobutton:

<input type="radio" ...>

Los controles de tipo radiobutton son similares a los controles de tipo checkbox, pero presentan una diferencia muy importante: **son mutuamente excluyentes**. Los radiobutton se utilizan cuando el usuario solamente puede escoger una opción entre las distintas opciones relacionadas que se le presentan. Cada vez que se selecciona una opción, automáticamente se deselecciona la otra opción que estaba seleccionaba.

El valor del atributo type para estos controles de formulario es **radio**. El atributo name se emplea para indicar los radiobutton que están relacionados. Por lo tanto, cuando varios radiobutton tienen el mismo valor en su atributo name, el navegador sabe que están relacionados y puede deseleccionar una opción del grupo de radiobutton cuando se seleccione otra opción.

Ejemplo

```
Sexo <br/>
<input type="radio" name="sexo" value="hombre" checked="checked" /> Hombre <br/>
<input type="radio" name="sexo" value="mujer" /> Mujer
```

Sexo
Hombre
Mujer

HTML
@mardeasa

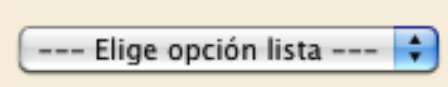
La etiqueta <select > </select>

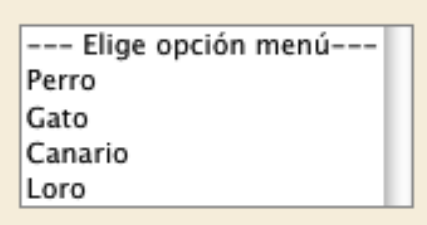
Genera un menú de opciones, con la posibilidad de visualizarse como una lista con deslizamiento o un menú desplegable.

Dispone de etiqueta de inicio y fin igual que textarea.

Atributos:

- size: Si el elemento <select> se presenta como una lista con desplazamiento ("scrolled list box"), este atributo especifica el número de opciones/elementos de la lista que deberían ser visibles al mismo tiempo.
- multiple: Atributo booleano que permite al usuario la posibilidad de seleccionar más de 1 opción de la lista, mediante la tecla control. Si no se especifica solo podrá seleccionar una.





La etiqueta <select > </select>

La etiqueta <select> creá el menu junto a la combinación de la etiqueta **<option>** y, opcionalmente la etiqueta **<optgroup>**. Ambos con etiquetas también de cierre.

- Todo elemento <select> debe contener al menos un elemento <option>
- Con <optgroup> permitimos agrupar opciones.

Atributos de la etiqueta <option ….>

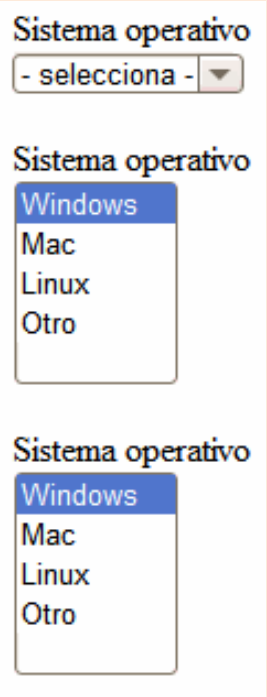
- **selected**: Indica que ésta opción es la preseleccionada inicialmente.
- **value**: Especifica el valor inicial de la opción a visualizarse. Si no se especifica el valor inicial es el texto incluido entre las etiquetas: <option> texto </option>. Obligatorio, ya que es el dato que se envía al Servidor cuando el usuario envía el formulario.
- **label**: Permite especificar un rótulo más corto para la opción.

La etiqueta <select > </select>

Ejemplo

```
<label for="so">Sistema operativo</label> <br>
    <select id="so" name="so">
      <option value="" selected>- selecciona -</option>
      <option value="windows">Windows</option>
      <option value="mac">Mac</option>
      <option value="linux">Linux</option>
      <option value="otro">Otro</option>
    </select>

<label for="so2">Sistema operativo</label> <br>
    <select id="so2" name="so2" size="5">
      <option value="windows" selected>Windows</option>
      <option value="mac">Mac</option>
      <option value="linux">Linux</option>
      <option value="otro">Otro</option>
    </select>

<label for="so3">Sistema operativo</label> <br>
    <select id="so3" name="so3" size="5" multiple="multiple">
      <option value="windows" selected>Windows</option>
      <option value="mac">Mac</option>
      <option value="linux">Linux</option>
      <option value="otro">Otro</option>
    </select>
```

Esta imagen muestra los tres tipos de listas desplegables disponibles.
El primero es el de las listas más utilizadas que sólo muestran un valor cada vez y sólo permiten seleccionar un valor.
El segundo tipo de lista es el que sólo permite seleccionar un valor pero muestra varios a la vez.
Por último, el tercer tipo de lista desplegable es aquella que muestra varios valores y permite realizar selecciones múltiples.



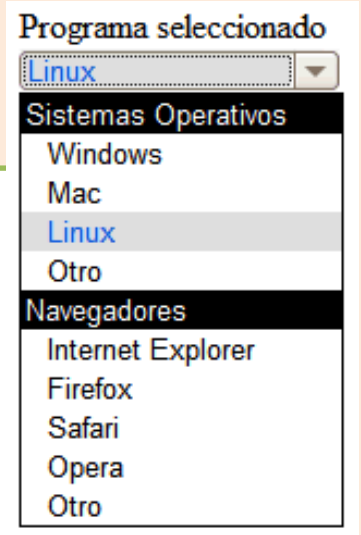
La etiqueta <select > </select>

Por otra parte, las listas desplegables permiten agrupar sus opciones de forma que el usuario pueda encontrar fácilmente las opciones cuando la lista es muy larga mediante la etiqueta **<optgroup>** .

El único atributo que suele utilizarse con la etiqueta <optgroup> es label, que indica el nombre de cada agrupación. Los navegadores muestran de forma destacada el título de cada agrupación, de forma que el usuario pueda localizar más fácilmente la opción deseada.

Ejemplo

```
<label for="programa">Programa seleccionado</label> <br>
<select id="programa" name="programa">
  <optgroup label="Sistemas Operativos">
    <option value="Windows" selected>Windows</option>
    <option value="Mac">Mac</option>
    <option value="Linux">Linux</option>
    <option value="Other">Otro</option>
  </optgroup>
  <optgroup label="Navegadores">
    <option value="Internet Explorer" selected>Internet Explorer</option>
    <option value="Firefox">Firefox</option>
    <option value="Safari">Safari</option>
    <option value="Opera">Opera</option>
    <option value="Other">Otro</option>
  </optgroup>
</select>
```



Botón de envío de formulario: <input type=”submit” ...>

La mayoría de formularios dispone de un botón para enviar al servidor los datos introducidos por el usuario:

<INPUT TYPE=“submit” NAME=“Nombre” VALUE=“Texto del boton”>

Ejemplo

```
<input type="submit" name="enviar" value="Enviar" />
```

Enviar

- El valor del atributo type para este control de formulario es submit. Provoca el envío del formulario para su procesamiento al recurso especificado en el atributo action de la etiqueta <form>.
- El navegador se encarga de enviar automáticamente los datos cuando el usuario pincha sobre este tipo de botón. El valor del atributo value es el texto que muestra el botón. Si no se establece el atributo value, el navegador muestra el texto predefinido *Enviar consulta*.

Boton de envío con imágenes:

<input type="image" ...>

El aspecto de los botones de formulario se puede personalizar por completo, ya que incluso es posible utilizar una imagen como botón.

El valor del atributo type para este control de formulario es image. El atributo src indica la URL de la imagen que debe mostrar el navegador en lugar del botón normal.

Ejemplo

```
<input type="image" name="enviar" src="disco.jpg">
```

Su principal ventaja es que permite personalizar por completo la estética de los botones y mostrarlos con un aspecto homogéneo en todos los navegadores. El principal inconveniente es que ralentiza la carga del formulario y que si se quiere modificar su aspecto, es necesario crear una nueva imagen.

Botón de reinicio de formulario:

<input type="reset" ...>

Aunque su uso era muy popular hace unos años, la mayoría de formularios modernos ya no utilizan este tipo de botón. Se trata de un botón especial que borra todos los datos introducidos por el usuario y devuelve el formulario a su estado original:

Ejemplo

```
<input type="reset" name="limpiar" value="Borrar datos del formulario" />
```

Borrar datos del formulario

El valor del atributo type para este control de formulario es reset. Como es habitual en los botones de formulario, el atributo value permite establecer el texto que muestra el botón. Si no se utiliza este atributo, el navegador muestra el texto predefinido del botón, que en este caso es Restablecer.

Botón de formulario:

<input type="button" ...>

Es un botón pulsador de uso genérico al que se pueden asociar distintos eventos que son manejados desde los scripts. Un uso muy frecuente consiste en asociar a la pulsación del botón la llamada a una función, por ejemplo, en Javascript, que compruebe la coherencia y la validez de los datos introducidos en el formulario para versiones anteriores a HTML5.

Ejemplo

```
<input type="button" name="botonAceptar" value="Aceptar" onclick="validaDatos()"/>
```

Aceptar

Información no visible: <input type="hidden" ..>

Los campos ocultos se emplean para añadir información oculta en el formulario:

Ejemplo

```
<input type="hidden" name="url_previa" value="/articulo/primero.html" />
```

El valor del atributo type para este control de formulario es hidden. Los campos ocultos no se muestran por pantalla, de forma que el usuario desconoce que el formulario los incluye. En las aplicaciones, a veces es preciso enviar información en los formularios sin que el usuario

tenga que introducir dicha información.

Por ejemplo, se puede desear incluir un campo con un número de sesión, pero que éste no sea visible por el usuario.

Envío de ficheros:

<input type="file" ...>

Los formularios también permiten adjuntar archivos para subirlos al servidor.

El valor del atributo type para este control de formulario es **file**. El navegador se encarga de mostrar un cuadro de texto donde aparece el nombre del archivo seleccionado y una ventana que permite navegar por los directorios y archivos del ordenador del usuario.

```
<input type="file" name="adjunto" />
```

Es obligatorio añadir el atributo **enctype** en la etiqueta <form> del formulario. El valor del atributo enctype debe ser **multipart/form-data**, por lo que la etiqueta <form> de los formularios que permiten adjuntar archivos siempre es:

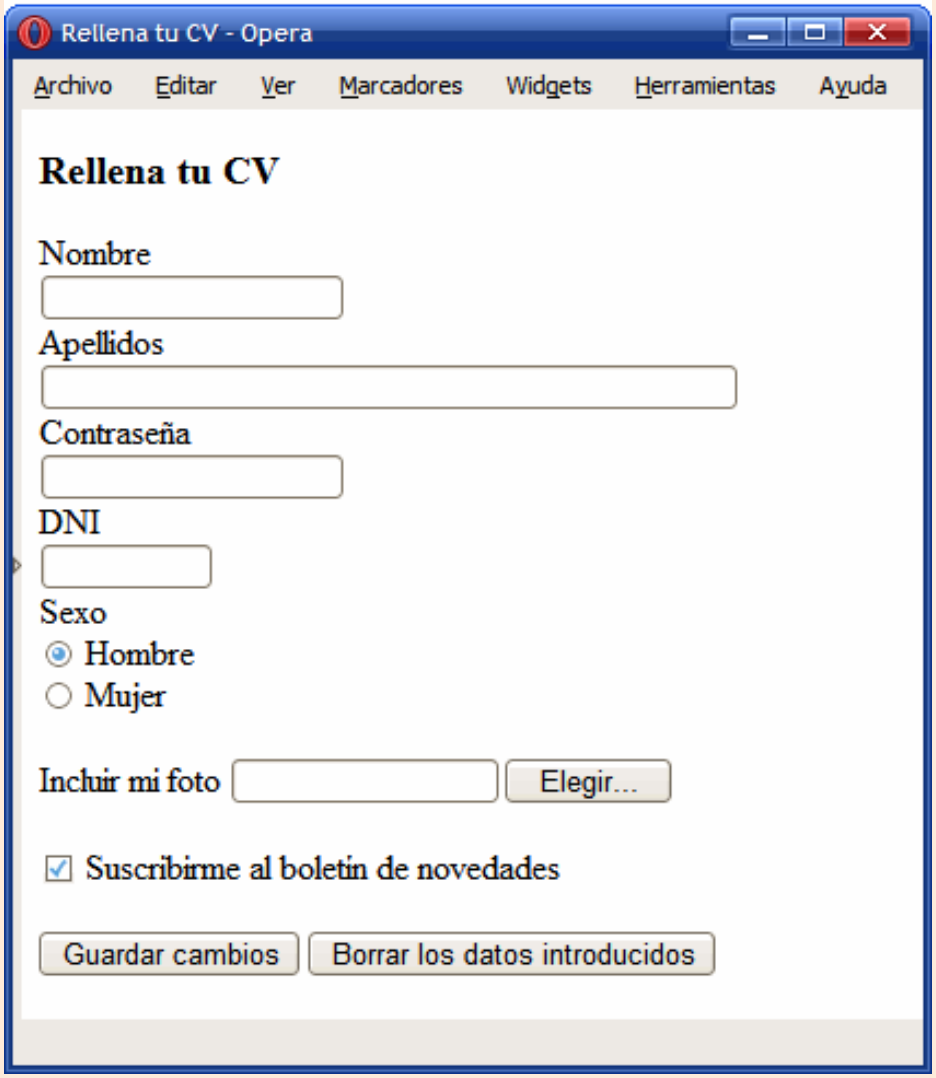
```
<form action="..." method="post" enctype="multipart/form-data">
```

Mediante el atributo **accept** podemos filtrar los tipos de archivos a subir, seprados por comas: un Tipo de medio de Internet; una extensión de archivo (como ".jpg" o ".pdf"); o una de las cadenas "audio/*", "video/*" e "image/*" representando archivos de sonido, video e imagen, respectivamente.

```
<input type="file" name="imagensubida" accept="image/png, .jpeg, .jpg, image/gif">
```

EJERCICIO

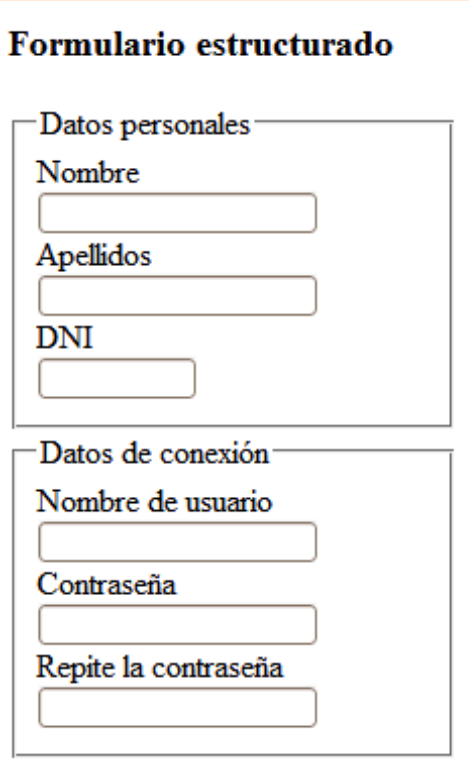
- Escribir el código HTML necesario para crear el formulario que se muestra en la imagen.
  1. Elegir el método más adecuado para el formulario (GET o POST) y cualquier otro atributo necesario.
  2. El nombre puede tener 30 caracteres como máximo, los apellidos 80 caracteres y la contraseña 10 caracteres como máximo.
  3. Asignar los atributos adecuados al campo del DNI.
  4. Por defecto, debe estar marcada la casilla de suscripción al boletín de novedades.



Formularios avanzados

Utilizando solamente las etiquetas <form> y <input> es posible diseñar la mayoría de formularios de las aplicaciones web. No obstante, HTML define algunos elementos adicionales para mejorar la estructura de los formularios creados.

La imagen muestra un formulario que agrupa sus elementos y añade etiquetas a cada campo para mejorar su estructura.



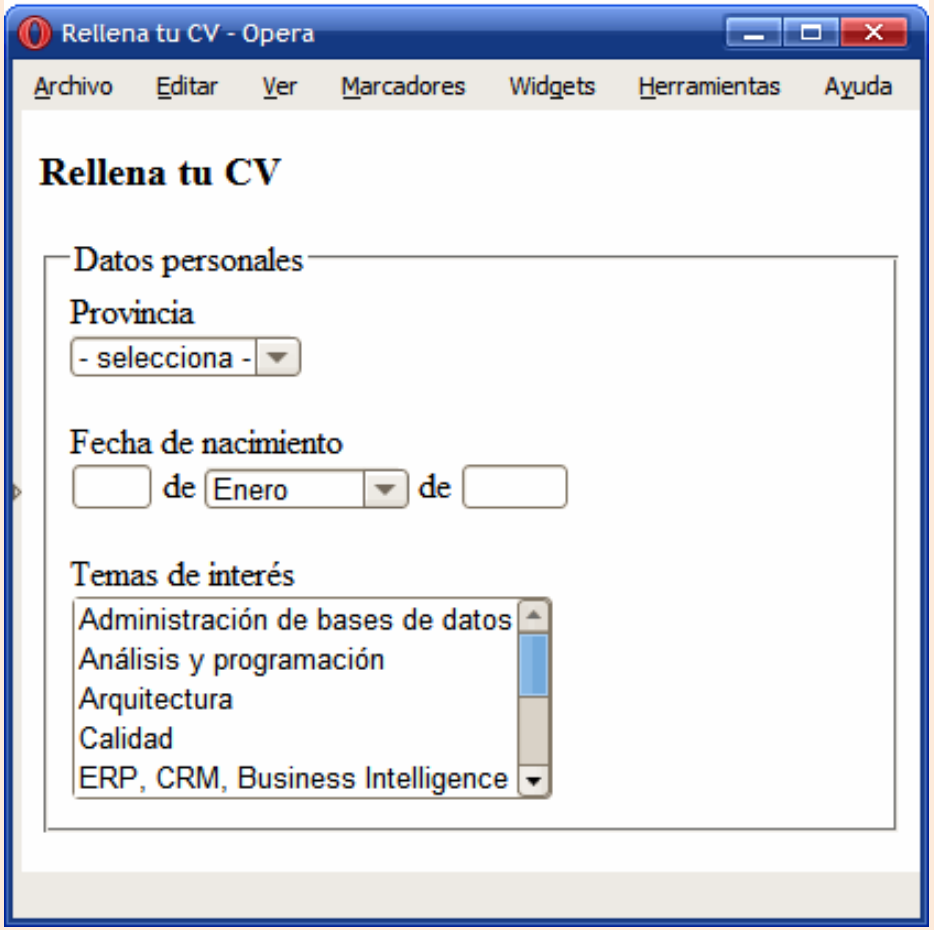
La etiqueta **<fieldset>** agrupa campos del formulario y la etiqueta **<legend>** asigna un nombre a cada grupo.

Podemos descargar el código HTML del formulario correspondiente a la imagen anterior y que hace uso de <fieldset> y <legend> para agrupar los campos del formulario.

La etiqueta <fieldset> agrupa todos los controles de formulario a los que encierra. El navegador muestra por defecto un borde resaltado para cada agrupación. La etiqueta <legend> se incluye dentro de cada etiqueta <fieldset> y establece el título que muestra el navegador para cada agrupación de elementos.

EJERCICIO

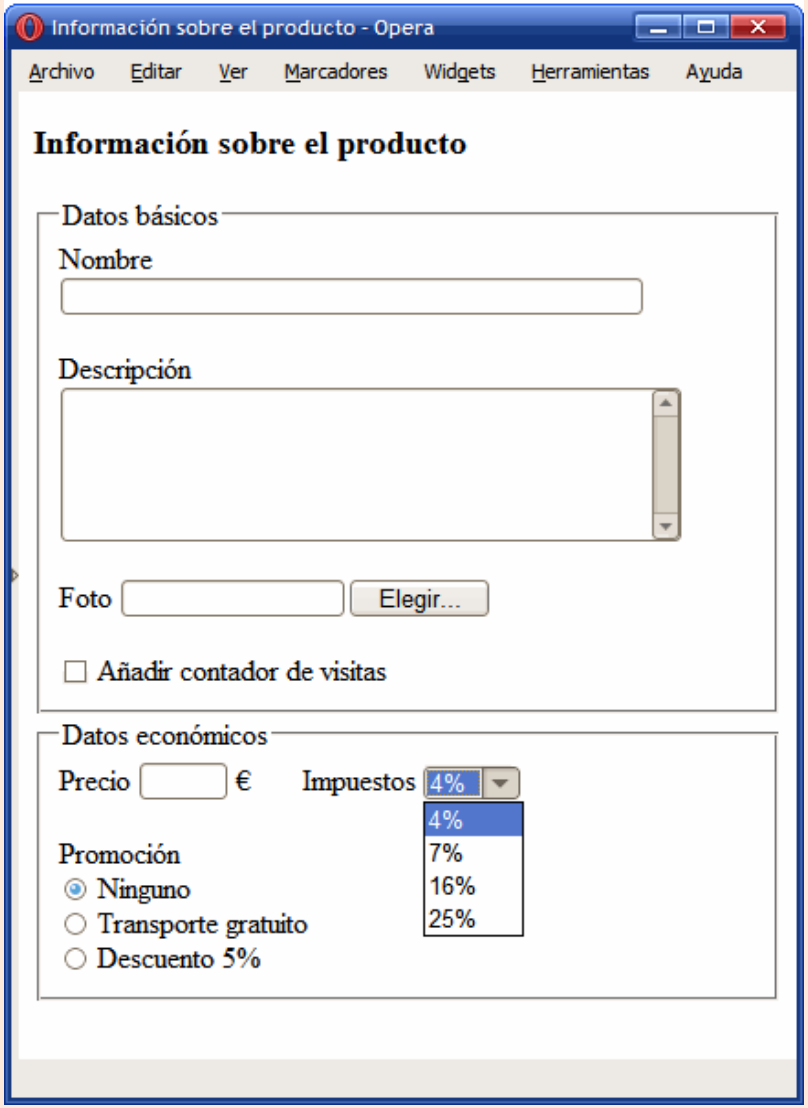
❖ Determinar el código HTML necesario para crear el formulario que se muestra en la siguiente imagen:



Solución

EJERCICIO

❖ Determinar el código HTML necesario para crear el formulario que se muestra en la siguiente imagen:



Solución

Nuevos atributos HTML5

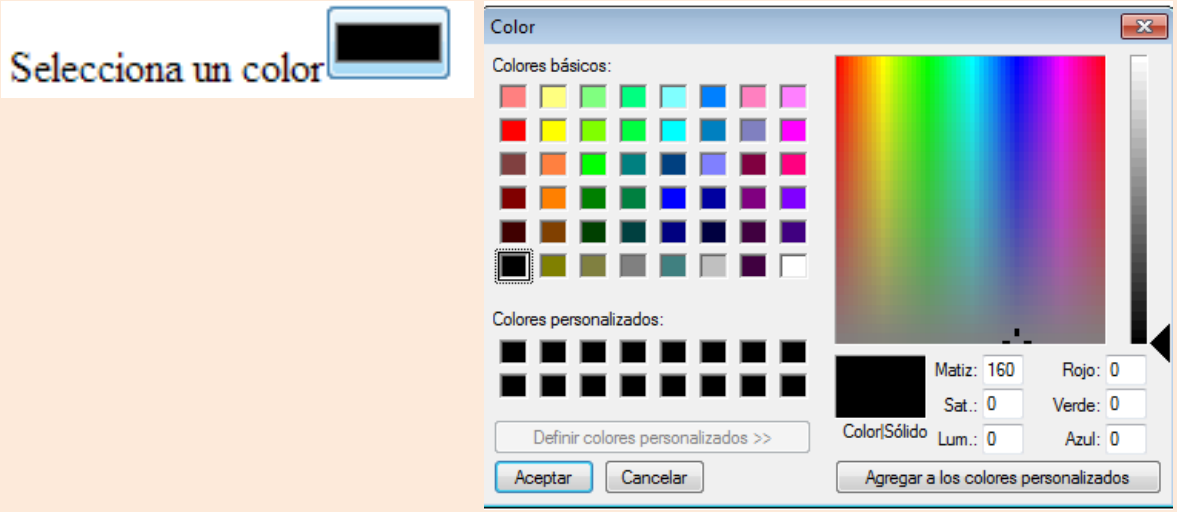
@mardeasa

Color:
<input type="color" ...>

Proporciona al usuario un selector de color, devuelve un valor RGB hexadecimal, como #FFF.

Ejemplo

```
Selecciona un color <input type="color" name="color" />
```



Selección de una Fecha con Date:

<input type="date" ...>

Permite al usuario seleccionar una fecha (año, mes, día) aunque no la hora.

Ejemplo

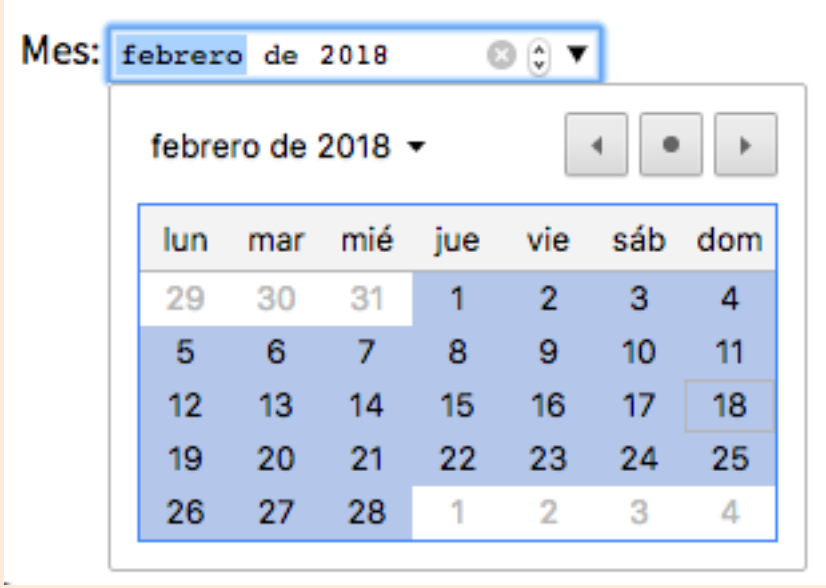
```
Fecha de inicio <input type="date" name="fechainic" />
```

Selección de un Mes:
<input type="month" ...>

Permite al usuario seleccionar un mes del año.

Ejemplo

```
Mes<input type="month" name="mes" />
```

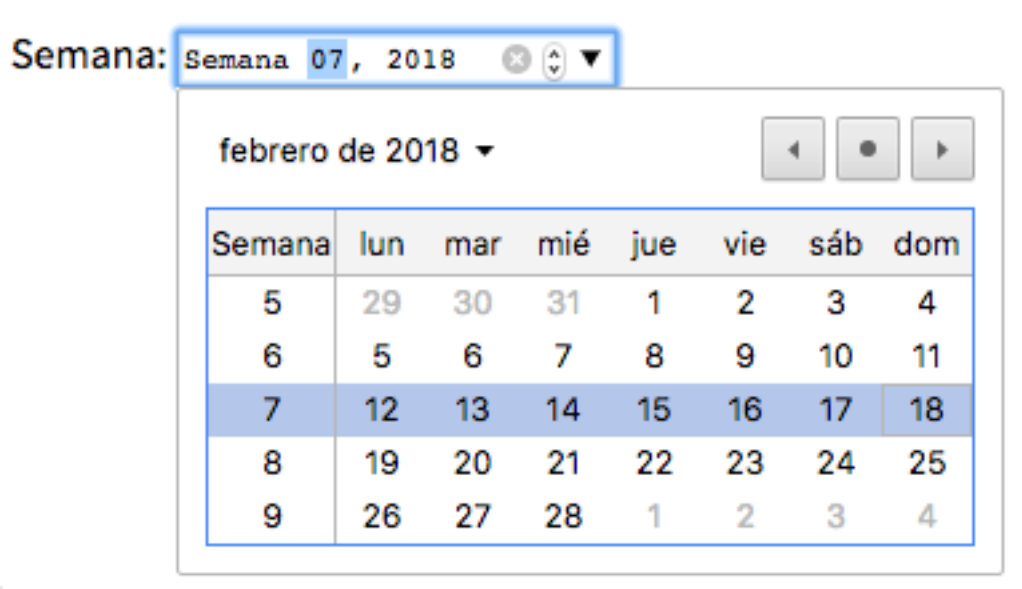


Selección de una Semana: <input type="week" ...>

Permite al usuario seleccionar una semana del año.

Ejemplo

```
Semana:<input type="week" name="semana" />
```



@mardeasa

Selección de una Fecha y hora con datetime:

<input type="datetime" ...>

Permite al usuario seleccionar una fecha (año, mes, día) y la hora incluye zona horaria. De momento no la soportan todos los navegadores.

Ejemplo

```
Fecha y hora <input type="datetime" name="fechahora" />
```

Selección de una Fecha y hora con datetime-local:

<input type="datetime-local" ...>

Permite al usuario seleccionar una fecha (año, mes, día) y la hora NO incluye zona horaria. De momento no la soportan todos los navegadores.

Ejemplo

```
Fecha y hora local sin zona horaria <input type="datetime-local" name="fechaHora" />
```

Selección de un número:

<input type="number" ...>

Proporciona una entrada de dato para introducir un número. Normalmente se trata de un cuadro de texto donde se puede escribir un número o hacer clic en las flechas arriba o abajo para seleccionarlo.

Tiene los atributos max y min para especificar los valores máximo y mínimo permitidos, también se puede utilizar el atributo step, este determina la cantidad a disminuir o aumentar cuando hace click en las flechas.

Ejemplos

```
Número <input type="number" name="num" />
```

Número 3

```
Número par del 0 al 10
<input type="number" max="10" min="0" step="2" name="par" />
```

Número par del 0 al 10 2

Selección de un rango de números:

<input type="range" ...>

Proporciona un deslizador en aquellos navegadores que lo soportan, al igual que el tipo number es soportado por Opera y los navegadores que trabajan con *Webkit*.
Al igual que number permite los atributos max, min y step.
La diferencia entre number y range, es que con range no es relevante obtener el valor exacto de un número, sino un rango numérico determinado.

Ejemplo

```
Seleccione un número <input type="range" name="num" />
```

HTML
@mardeasa

Selección de una búsqueda: <input type="search" ...>

Proporciona un campo de búsqueda, es muy parecido a un campo de texto normal. Algunos navegadores agregan el icono ✕ para borrar el contenido de la búsqueda.

Ejemplo

```
Buscar <input type="search" name="busqueda" placeholder="buscar…" />
<input type="submit" value="IR" />
```

Buscar buscar .. IR

Control para el email:
<input type="email" ...>

Se utiliza para especificar una o más direcciones de correo electrónico.

Soporta el atributo booleano multiple, para permitir varias direcciones separadas por comas.

Sigue pareciendo un campo de texto normal, sin embargo, si se intenta enviar el formulario y los datos introducidos no son formato de email, algunos navegadores ofrecen un mensaje de error.

Ejemplo

```
Correo electrónico
<input type="email" name="email" placeholder="user@dominio.com" />
```

Correo electrónico user@dominio.com

Selección de un teléfono:

<input type="tel" ...>

Para pedir un número de teléfono al usuario usaremos el tipo de entrada "tel". A diferencia de los tipos url y email, el tipo tel no impone su sintaxis o patrón determinado. Tanto las letras como los números son válidos. De todas formas, podemos crear nosotros mismos el formato que nos interese utilizando patrones o Javascript.

Ejemplo

```
Teléfono (9 dígitos)
<input type="tel" name="tlf" pattern="|^\d{9}$|" placeholder="teléfono" />
```

Telefono (9 digitos) telefono

Control para el url:
<input type="url" ...>

Para pedir al usuario una entrada de una dirección web. Se muestra como un simple campo de texto normal. En muchas pantallas táctiles, el teclado se optimizará para la entrada de direcciones web, con una barra(/) y una tecla de método abreviado (.com).

HTML 5 no valida que realmente sea una url válida, si quieres proporcionar a tu formulario un patrón de url utiliza el atributo pattern="regexp" (expresión regular) http://html5pattern.com/

```
URL <input type="url" name="url" />
```

URL http://mardeasa.es

La etiqueta <datalist> </datalist>

Permite automatizar la introdución de datos.

Genera una lista desplegable de opciones para que el usuario realice su selección.

La ventaja de este nuevo elemento es que reconoce los primeros caracteres introducidos por el usuario, para “autocompletar” la información.

Ejemplo

```
<label for= "pais"> Selecciona tu país: </label>
<input type= "text" name= "pais" id= "pais " list="paises" />
<datalist id="paises">
         <option value="España"></option>
         <option value="Italia"></option>
         <option value="Portugal"></option>
         <option value="Grecia"></option>
</datalist>
```

La etiqueta <keygen>

Esta etiqueta genera 2 claves, una pública y una privada.

Cuando se envía el formulario, la clave privada se almacena localmente y sólo la clave pública es enviada al Servidor.

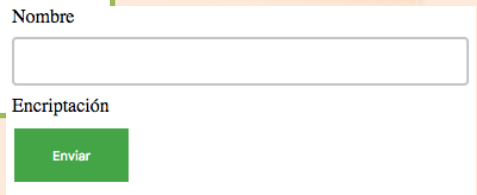
Es un elemento de seguridad útil para instituciones que deben validar el acceso de usuarios a información privada desde el exterior de la red.

Los atributos de esta etiqueta son:

- **challenge**: genera una serie de caracteres de validación que se envían junto con la clave.
- **keytype**: especifica el tipo de algoritmo de seguridad utilizado por la clave. Por defecto, usa RSA.
- **name**: nombre asociado a keygen.
- **autofocus**: asigna el focus al cargar la página.
- **form**: nombre del formulario al cual pertenece esta etiqueta
- **disabled**: atributo booleano, si su valor es true, indica que la etiqueta esta deshabilitada

```
<form action="demo_keygen.php" method="get">
        Nombre <input type= "text" name= "nombre" /> <br>
        Encriptación <keygen name="security"><br>
        <input type="submit" />
</form>
```

Ejemplo



Validar formularios en HTML5

Atributos básicos de validación

En nuestros campos de entrada de datos, se pueden utilizar ciertos atributos para realizar validaciones sencillas. Algunos de estos atributos ya los hemos visto en apartados anteriores, sin embargo, vamos a comentarlos:

Validar formularios en HTML5

Atributo	Valor	Puede actuar sobre...	Descripción
minlength	*número*	Campos de texto	Establece la longitud mínima del texto requerida.
maxlength	*número*	Campos de texto	No permite escribir textos superiores a *número* carácteres.
min	*número*	Campos numéricos	Establece el número mínimo permitido.
	fecha	Campos de fecha	Establece la fecha mínima permitida.
	hora	Campos de hora	Establece la hora mínima permitida.
max	*número*	Campos numéricos	Establece el número máximo permitido.
	fecha	Campos de fecha	Establece la fecha máxima permitida.
	hora	Campos de hora	Establece la hora máxima permitida.
step	*número*	Campos numéricos	Establece el salto de números permitido. Por defecto, 1.
	fecha	Campos de fecha	Establece el salto de días permitido. Por defecto, 1.
	hora	Campos de hora	Establece el salto de segundos permitido. Por defecto, 1.
required		Campos en general	Campo obligatorio. Se debe rellenar para enviar formulario.
disabled		Campos en general	Campo desactivado. No se puede modificar. No se envía.
readonly		Campos en general	Campo de sólo lectura. No se puede modificar. Se envía.

Validar formularios en HTML5

Atributos básicos de validación

Con estos atributos, podemos crear validaciones básicas en nuestros campos de entrada de datos, obligando al usuario a rellenar un campo obligatorio, forzando a indicar valores entre un rango numérico o permitiendo sólo textos con un tamaño específico, entre otros.

Además, utilizando las **pseudoclases** CSS de validación **:valid** e **:invalid** podemos aplicar estilos a los campos <input> y <textarea> teniendo en cuenta su validación.

HTML

```
<form name="formulario" method="post" action="http://pagina.com/send.php">
  <!-- Nombre de usuario. Campo obligatorio, entre 5-40 carácteres -->
  Usuario:
<input type="text" name="nombre" placeholder="Por ejemplo, Juan" minlength="5" maxlength="40" required>
  <!– Contraseña. Campo obligatorio, mínimo 11 carácteres -->
  Password: <input type="password" name="pass" placeholder="Contraseña" minlength="11" required>
</form>
```

CSS

```
input:valid, textarea:valid {  background:green;}
input:invalid, textarea:invalid {  background:red;}
```

Validar formularios en HTML5 patrones: pattern

Ejemplos de patrones HTML5

Para clarificar estos conceptos y entenderlos mejor, vamos a mostrar algunos ejemplos de campos con validaciones HTML5 en casos reales, y así afianzar conceptos sobre expresiones regulares y su aplicación en los atributos **pattern**:

- Tipo de campo: Nombre de usuario
- Campo obligatorio: required.
- Entre 5-40 carácteres: minlength="5" maxlength="40"
- Sólo se permiten letras (mayúsculas y minúsculas) y números: **pattern="[A-Za-z0-9]+"**

```
<form name="formulario" method="post" action="http://pagina.com/send.php">
        <!-- Nombre de usuario. Campo obligatorio, entre 5-40 caracteres y
        Sólo se permiten letras y números -->
        <input type="text" name="nombre" placeholder="Su nombre de usuario"
         minlength="5" maxlength="40" required pattern="[A-Za-z0-9]{5,40}"
        title="Letras y números. Tamaño mínimo: 5. Tamaño máximo: 40">
</form>
```

Validar formularios en HTML5
patrones: pattern

Expresión regular	Carácter especial	Denominación	Descripción
`.`	Punto	Comodín	Cualquier carácter (o texto de tamaño 1)
`A|B`	*Pipe*	Opciones lógicas	Opciones alternativas (o `A` o `B`)
`C(A|B)`	Paréntesis	Agrupaciones	Agrupaciones alternativas (o `CA` o `CB`)
`[0-9]`	Corchetes	Rangos de carácteres	Un dígito (del `0` al `9`)
`[A-Z]`			Una letra mayúscula de la `A` a la `Z`
`[^A-Z]`	`^` en corchetes	Rango de exclusión	Una letra que no sea mayúscula de la `A` a la `Z`
`[0-9]*`	Asterisco	Cierre o clausura	Un dígito repetido 0 ó más veces (vacío incluido)
`[0-9]+`	Signo más	Cierre positivo	Un dígito repetido 1 ó más veces
`[0-9]{3}`	Llaves	Coincidencia exacta	Cifra de 3 dígitos (dígito repetido `3` veces)
`[0-9]{2,4}`		Coincidencia (rango)	Cifra de 2 a 4 dígitos (rep. de `2` a `4` veces)
`b?`	Interrogación	Carácter opcional	El carácter `b` puede aparecer o puede que no
`\.`	Barra invertida	Escape	El carácter `.` literalmente (no como comodín)

Validar formularios en HTML5 patrones: pattern

Patrones de validación HTML5

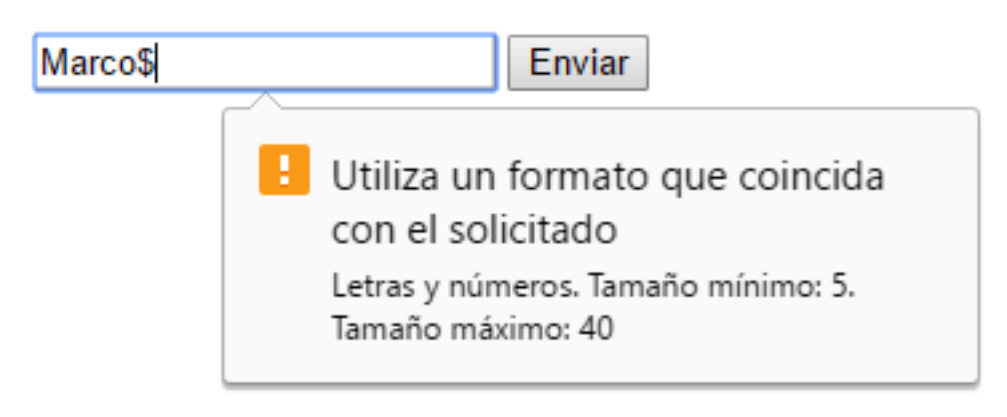
No obstante, aunque los atributos de validación básicos son muy interesantes y pueden facilitarnos la tarea de validación, en muchos casos son insuficientes. Para ello tenemos los patrones de validación HTML5, mucho más potentes y flexibles, que nos permitirán ser mucho más específicos utilizando expresiones regulares para validar datos.

Una expresión regular es una cadena de texto que representa un posible patrón de coincidencias, que aplicaremos mediante el atributo pattern en los campos que queramos validar.

Para ello hay que conocer algunas características básicas de las expresiones regulares:

Validar formularios en HTML5

En el ejemplo anterior, no se limitará al usuario a la hora de escribir, como hace maxlength, sino que permitirá al usuario escribir la información que desee y en caso de no pasar la validación, mostrará un mensaje de advertencia y no lo dejará continuar hasta que termine. Además, hemos ampliado el mensaje de advertencia incluyendo el texto en el atributo title.



Validar formularios en HTML5

En el siguiente caso, se pide al usuario que indique el modelo de coche que posee, en un posible formulario de servicio técnico. Los modelos posibles son A1, A3, A4 y A15. En lugar de mostrar una lista de selección, podemos mostrar un campo de texto y colocar una validación como la siguiente:

- Tipo de campo: Modelo de coche
- Campo obligatorio: required.
- Sólo se permiten las opciones: A1, A3, A4 y A15

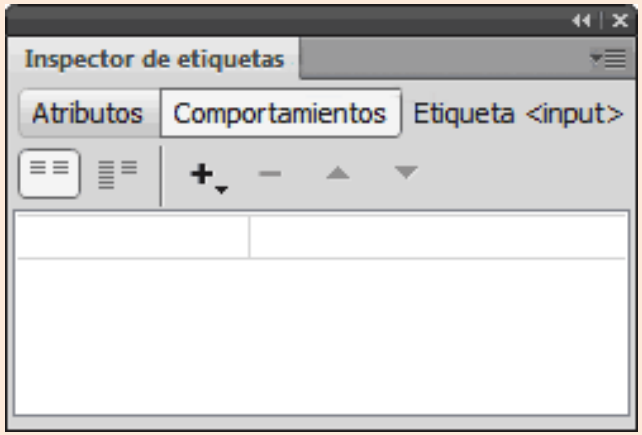
```
<form name="formulario" method="post" action="http://pagina.com/send.php">
 <!-- Modelo de coche Campo obligatorio, opciones posibles: A1, A3, A4 y A15
    Sólo se permiten letras y números -->
 <input type="text" name="coche" placeholder="Su modelo de coche"
      required pattern="A(1|3|4|15)|a(1|3|4|15)" title="Modelos posibles: A1, A3,
A4 y A15">
</form>
```

Validar formularios en HTML4 con Dreamweaver

La validación de formularios sirve para hacer que JavaScript valide el formulario antes de que se envíe, avisando al usuario para que corrija los errores, como campos obligatorios sin rellenar. Esto es mucho más eficiente y rápido que enviar la página y validarla sólo en el servidor.

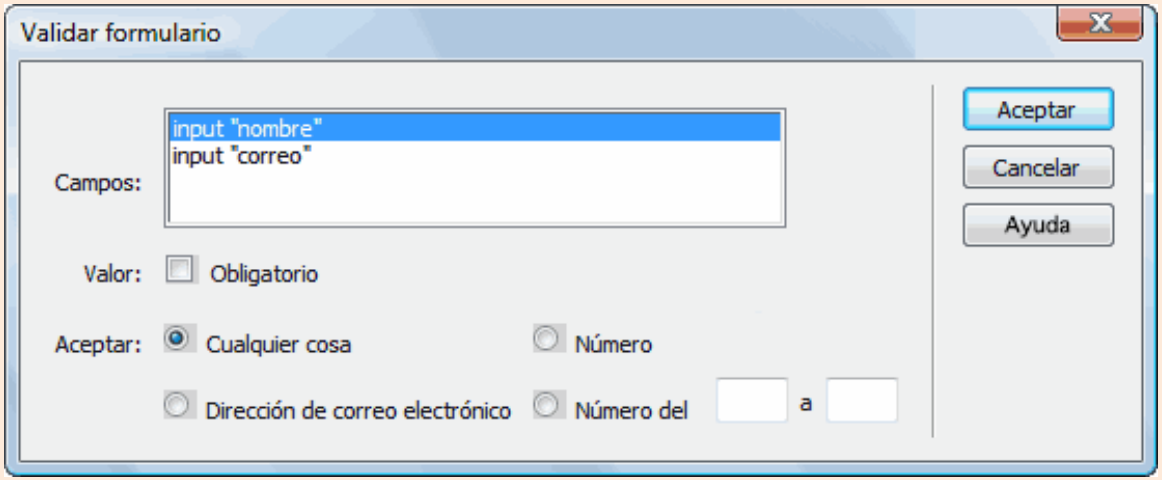
Para validar un formulario desde Dreamveaver, hay que abrir el panel de Comportamientos. Este panel se puede abrir a través del menú Ventana, opción Comportamientos, o pulsando Mayús + F4. Comportamientos forma parte del panel Inspector de etiquetas.

En este panel hay que desplegar el botón + y pulsar sobre la opción Validar formulario, deberás haber seleccionado el formulario previamente.



Validar formularios en HTML4 con Dreamweaver

Entonces se mostrará una ventana como la siguiente, donde aparecen todos los elementos del formulario.



Puede seleccionarse uno por uno cada elemento del formulario, pudiendo especificar los requisitos que ha de cumplir.

Puede establecerse como campo a rellenar obligatoriamente (Valor Obligatorio), y si su contenido ha de ser numérico (Número) y si ha de estar en un rango, una Dirección de correo electrónico, etc.

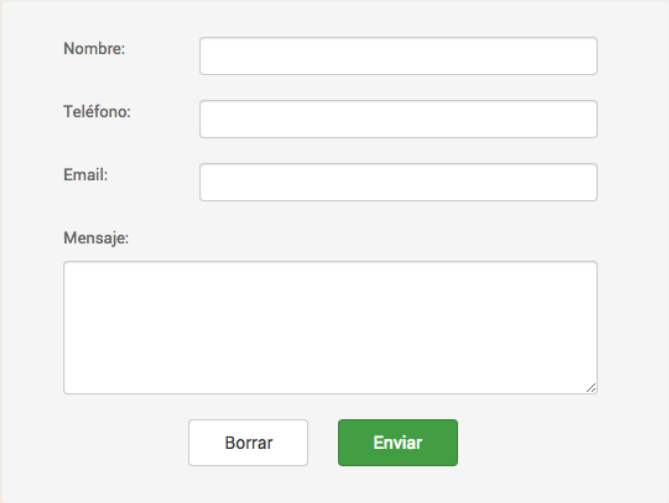
Envío de formularios

Si queremos incluir un formulario de contacto en nuestra página web tenemos que tener en cuenta que el atributo action trabaja con el lenguaje de programación PHP, por lo tanto:

- Sólo funcionará si nuestra web está alojada en un servidor que lo soporte (tenéis que ver las características de vuestro alojamiento).
- Si trabajáis en un servidor local (en vuestra máquina con Wampserver o MAMP), tenéis que tener instalado un servidor smtp, para que envíe el email.
- Necesitáis tener unos conocimientos mínimos de PHP. Al menos, saber que el archivo en el que queráis que se ejecute el php tiene que tener extensión .php (no .html), por ejemplo: formulario.php.

Envío de formularios

Teniendo en cuenta estos tres puntos básicos, ya podemos ver el código completo de nuestro archivo de formulario HTML y del envío del formulario PHP:




```php
<?php
$nombre = $_POST['nombre'];
$mail = $_POST['mail'];
$empresa = $_POST['empresa'];

$header = 'From: ' . $mail . " \r\n";
$header .= "X-Mailer: PHP/" . phpversion() . " \r\n";
$header .= "Mime-Version: 1.0 \r\n";
$header .= "Content-Type: text/plain";

$mensaje = "Este mensaje fue enviado por " . $nombre . ",
 de la empresa " . $empresa . " \r\n";
$mensaje .= "Su e-mail es: " . $mail . " \r\n";
$mensaje .= "Mensaje: " . $_POST['mensaje'] . " \r\n";
$mensaje .= "Enviado el " . date('d/m/Y', time());

$para = 'santiagomartindejesus@gmail.com';
$asunto = 'Datos del formulario: ';

mail($para, $asunto, utf8_decode($mensaje), $header);

echo 'El mensaje fue enviado correctamente';

//header('Location: http://mardeasa.es');
?>
```

Envío de formularios

Un poco de explicación:

El atributo action de la etiqueta form indica a donde tiene que dirigirse a la hora de enviar el formulario, con el código puesto simplemente le indicamos que recargue la página.

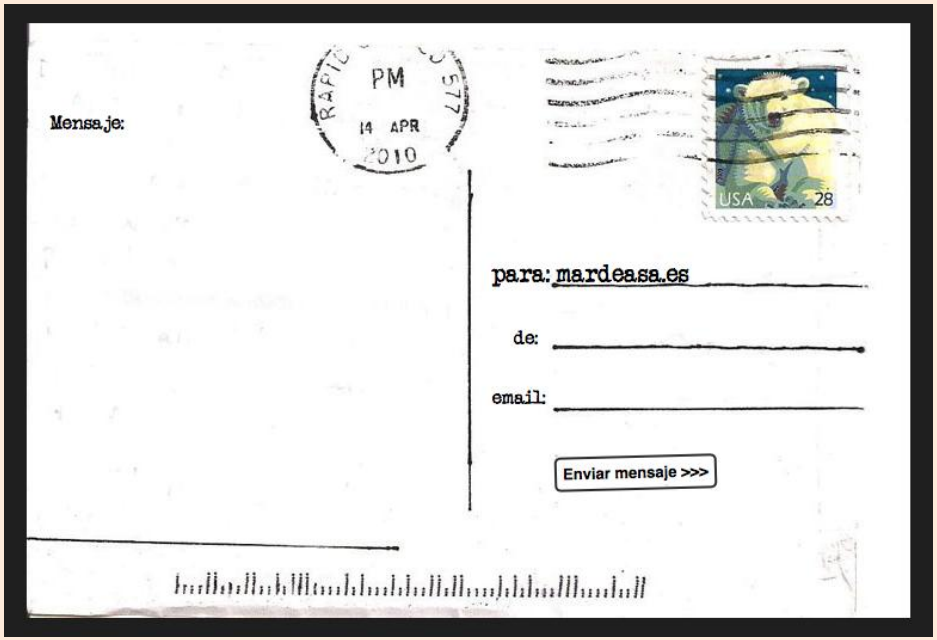
Con la variable $_POST recogemos los datos del formulario que escribió el usuario y los enviamos, el nombre de cada variable corresponde con el name de cada input.

La función MAIL es la encargada de, con todos los datos, enviar el formulario, las variables que necesita son:

- Email de destino
- Asunto del mensaje
- Mensaje
- Remitente

Con esto ya debería funcionar, tenéis que acordaros de cambiar el email de la variable destino con el email al que queráis que lleguen los mensajes.

Algunos servidores pueden reconocer como span los correos enviados mediante la función MAIL, lo que impide que lleguen al destino.



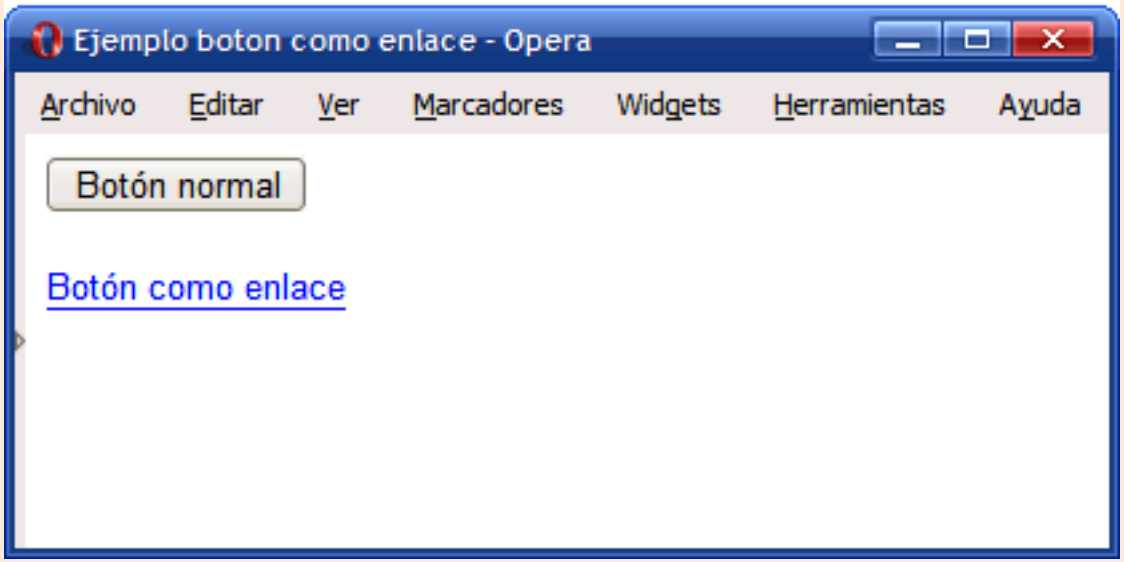
Estilos en formularios

@mardeasa

CSS en Formularios

Mostrar un botón como un enlace

El estilo por defecto de los enlaces se puede modificar para que se muestren como botones de formulario. Ahora, los botones de formulario también se pueden modificar para que parezcan enlaces.



CSS en Formularios

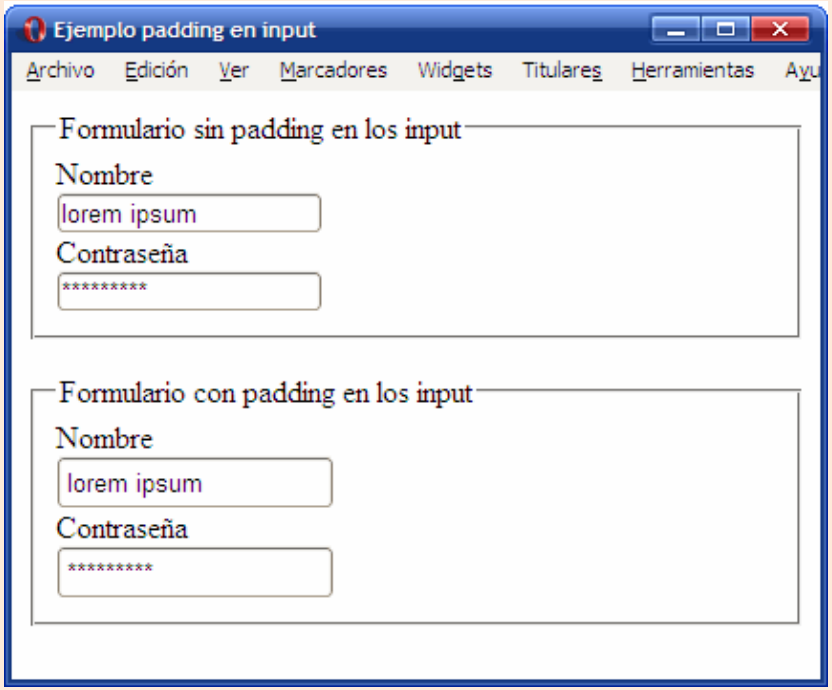
Mostrar un botón como un enlace

Las reglas CSS del ejemplo anterior son las siguientes:

```
.enlace {
  border: 0;
  padding: 0;
  background-color: transparent;
  color: blue;
  border-bottom: 1px solid blue;
}
 -------------------------------

<input class="enlace" type="button" value="Botón como enlace" />
```

CSS en Formularios

Mejoras en los campos de texto



Por defecto, los campos de texto de los formularios no incluyen ningún espacio de relleno, por lo que el texto introducido por el usuario aparece pegado a los bordes del cuadro de texto.

Añadiendo un pequeño padding a cada elemento <input>, se mejora notablemente el aspecto del formulario.

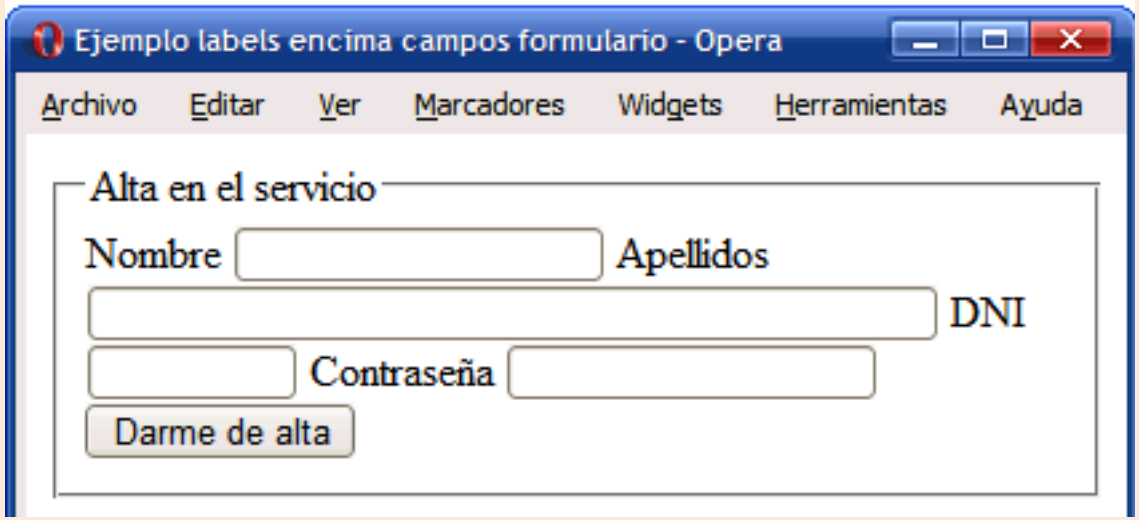
La regla CSS necesaria para mejorar el formulario es muy sencilla:

```
form.elegante input {
  padding: .2em;
}
```

CSS en Formularios

Labels alineadas y formateadas

Los elementos <input> y <label> de los formularios son elementos en línea, por lo que el aspecto que muestran los formularios por defecto, es similar al de la siguiente imagen:



CSS en Formularios

Labels alineadas y formateadas

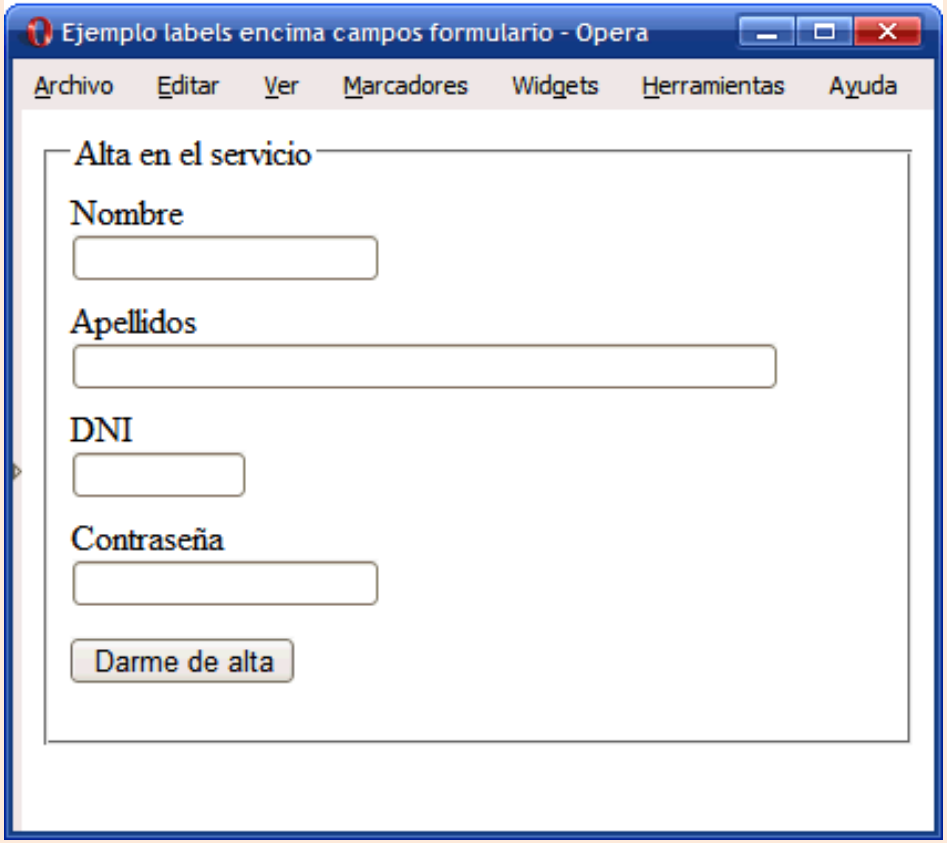
El código HTML del ejemplo anterior es el siguiente:

```
<form>
<fieldset>
  <legend>Alta en el servicio</legend>

  <label for="nombre">Nombre</label>
  <input type="text" id="nombre" />

  <label for="apellidos">Apellidos</label>
  <input type="text" id="apellidos" size="50" />

  <label for="dni">DNI</label>
  <input type="text" id="dni" size="10" maxlength="9" />

  <label for="contrasena">Contraseña</label>
  <input type="password" id="contrasena" />

  <input class="btn" type="submit" value="Darme de alta" />
</fieldset>
</form>
```

CSS en Formularios

Labels alineadas y formateadas

Aprovechando los elementos <label>, se pueden aplicar unos estilos CSS sencillos que permitan mostrar el formulario con el aspecto de la siguiente imagen:



CSS en Formularios

Labels alineadas y formateadas

En primer lugar, se muestran los elementos <label> como elementos de bloque, para que añadan una separación para cada campo del formulario. Además, se añade un margen superior para no mostrar juntas todas las filas del formulario:

```
label {
  display: block;
  margin: .5em 0 0 0;
}
```

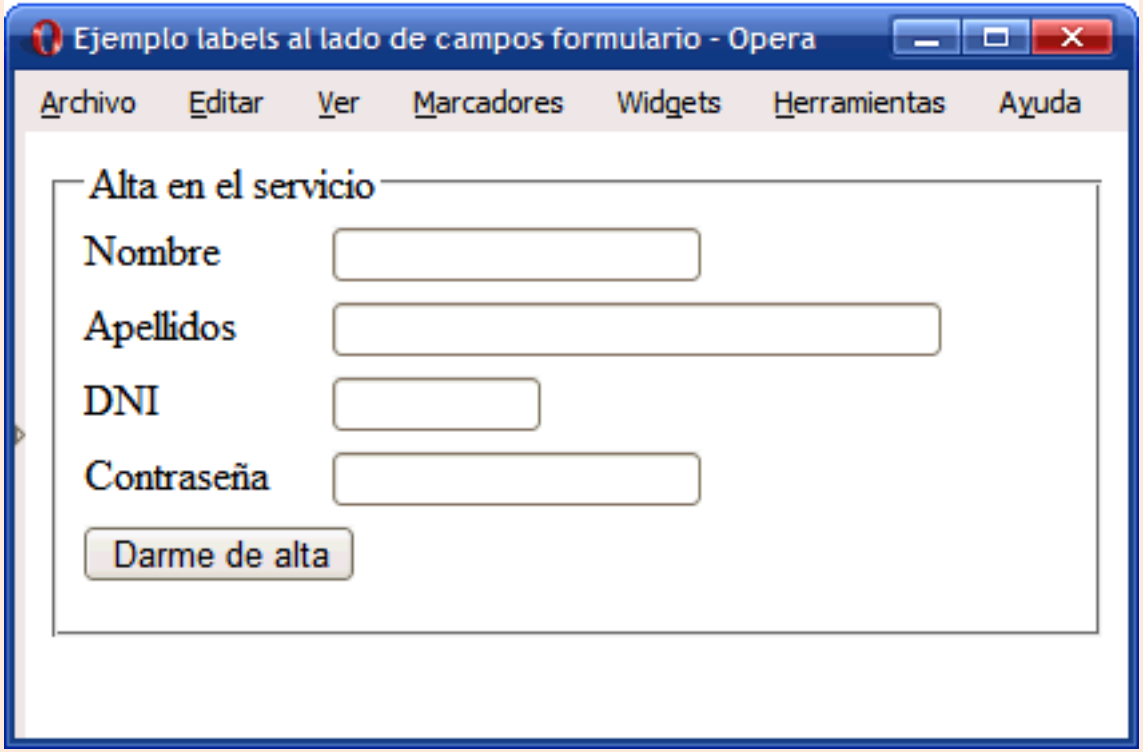
El botón del formulario también se muestra como un elemento de bloque y se le añade un margen para darle el aspecto final deseado:

```
.btn {
  display: block;
  margin: 1em 0;
}
```

CSS en Formularios

Labels alineadas y formateadas

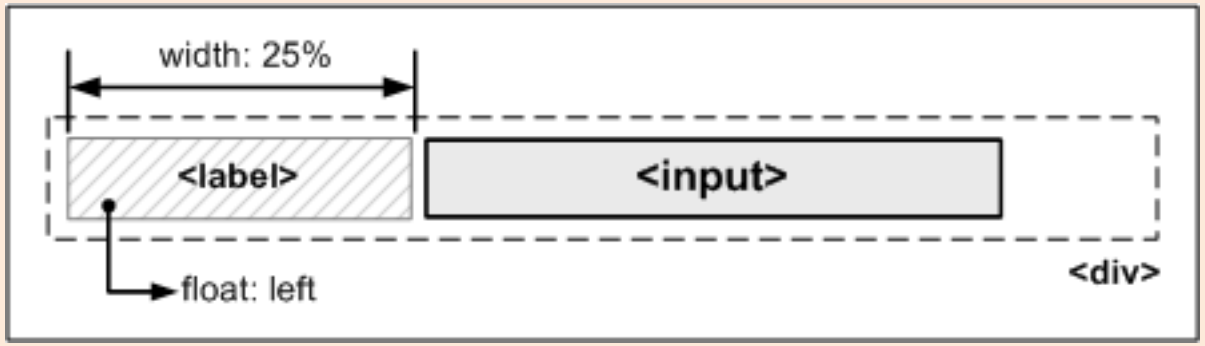
En ocasiones, es más útil mostrar todos los campos del formulario con su <label> alineada a la izquierda y el campo del formulario a la derecha de cada <label>, como muestra la siguiente imagen:



CSS en Formularios

Labels alineadas y formateadas

Para mostrar un formulario tal y como aparece en la imagen anterior no es necesario crear una tabla y controlar la anchura de sus columnas para conseguir una alineación perfecta. Sin embargo, sí que es necesario añadir un nuevo elemento (por ejemplo un) que encierre a cada uno de los campos del formulario (<label> y <input>). El esquema de la solución propuesta es el siguiente:



CSS en Formularios

Labels alineadas y formateadas

Por tanto, en el código HTML del formulario anterior se añaden los elementos :

```
<form>
  <fieldset>
  <legend>Alta en el servicio</legend>

  <div>
    <label for="nombre">Nombre</label>
    <input type="text" id="nombre" />
  </div>

  <div>
    <label for="apellidos">Apellidos</label>
    <input type="text" id="apellidos" size="35" />
  </div>
  ...
</fieldset>
</form>
```

CSS en Formularios

Labels alineadas y formateadas

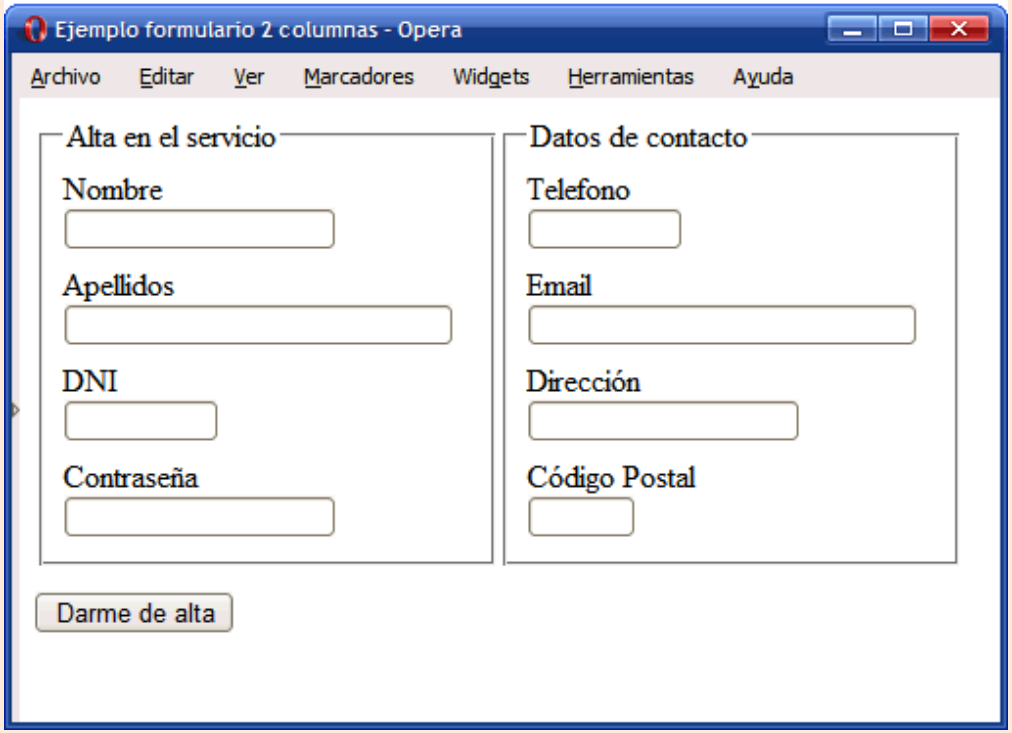
Y en el código CSS se añaden las reglas necesarias para alinear los campos del formulario:

```
div {
  margin: .4em 0;
}
div label {
  width: 25%;
  float: left;
}
```

CSS en Formularios

Formulario en varias columnas

Los formularios complejos con decenas de campos pueden ocupar mucho espacio en la ventana del navegador. Además del uso de pestañas para agrupar los campos relacionados en un formulario, también es posible mostrar el formulario a dos columnas, para aprovechar mejor el espacio.



CSS en Formularios

Formulario en varias columnas

La solución consiste en aplicar la siguiente regla CSS a los <fieldset> del formulario:

```
form fieldset {
  float: left;
  width: 48%;
}

<form>
  <fieldset>
    ...
  </fieldset>

  ...
</form>
```

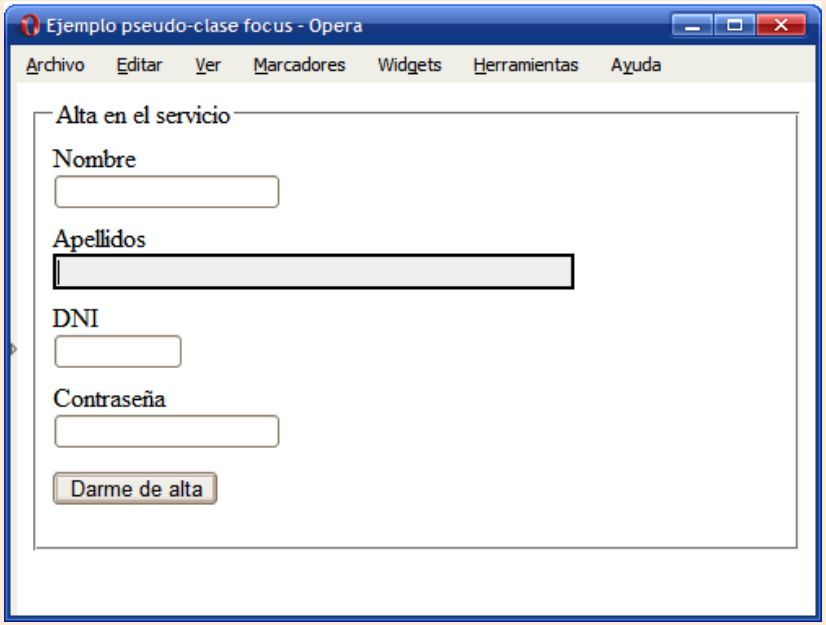
Si se quiere mostrar el formulario con más de dos columnas, se aplica la misma regla pero modificando el valor de la propiedad width de cada <fieldset>. Si el formulario es muy complejo, puede ser útil agrupar los <fieldset> de cada fila mediante elementos .

CSS en Formularios

Resaltar el campo seleccionado

Una de las mejoras más útiles para los formularios HTML consiste en resaltar de alguna forma especial el campo en el que el usuario está introduciendo datos. Para ello, CSS define la pseudo-clase :focus, que permite aplicar estilos especiales al elemento que en ese momento tiene el foco o atención del usuario.

La siguiente imagen muestra un formulario que resalta claramente el campo en el que el usuario está introduciendo la información:



CSS en Formularios

Resaltar el campo seleccionado

Añadiendo la pseudo-clase :focus después del selector normal, el navegador se encarga de aplicar esos estilos cuando el usuario activa el elemento:

```
input:focus {
  border: 2px solid #000;
  background: #F3F3F3;
}
```

CSS en Formularios

Pseudoclases :valid e :invalid

La pseudo-clase **:valid** CSS representa cualquier elemento de tipo <input> o <form> que cumpla con los ajustes de validación especificados para dicho elemento. De esa forma podemos especificar un estilo y apariencia que confirme al usuario que los datos introducidos en el control de formulario son correctos.

```
input:valid {background-color: #ddffdd;}
```

Igualmente sucede con la pseudo-clase **:invalid**, que representa cualquier elemento de tipo <input> o <form> que no cumpla con los ajustes de validación especificados para dicho elemento.

```
input:invalid {background-color: #ffdddd;}
form:invalid {border: 5px solid #ffdddd;}
```

CSS en Formularios

Ejemplo

Pseudoclases :valid e :invalid

Utilizando estas pseudoclases junto con fontawesome podemos realizar el siguiente ejemplo para la verificación de formularios, en el que incluye un símbolo de verificación al final del campo que sea validado correctamente:

Incluimos dentro del <head>:

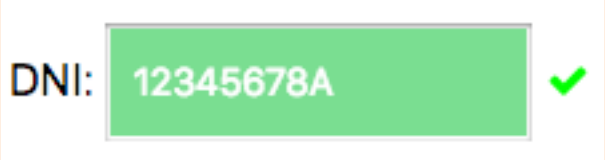
```
<link  href="http://maxcdn.bootstrapcdn.com/font-awesome/4.2.0/css/
font-awesome.min.css" rel="stylesheet">
```

En el CSS aplicamos las siguientes reglas:

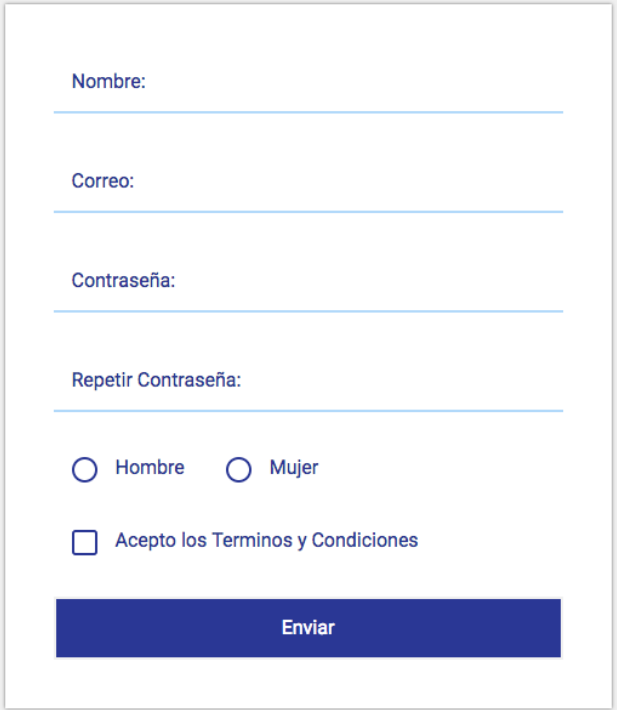
```
.check-ok {color: lime;}
input:invalid ~ .check-ok {display: none;}
input:valid ~ .check-ok {display: inline; margin-left: 5px;}
```

Y después de cada input que queramos verificar:

```
<i class="fa fa-check check-ok"></i>
```



Ejemplos de aplicación de estilos a formularios



EVENTOS

Un evento es algo que sucede en el navegador, de tal manera que, cuando ocurre, podemos definir algún comportamiento concreto, una respuesta por parte de la página web, mediante nuestro medio o utilizando otras tecnologías del lado del servidor.

Hemos dejado esta sección para el final del curso puesto que en HTML no podemos hacer más que definirlos, ya que para programarlos es necesario usar el lenguaje de programación JavaScript.

A continuación describiremos los atributos usados para asignar eventos en HTML:

EVENTOS

Eventos de ventana:

onload: se produce al cargarse el contenido de la página web.

onunload: se lanza al cargar otro documento o página web.

Eventos de formularios:

onblur: cuando un elemento pierde el foco.

onchange: al modificar un elemento.

onfocus: cuando un elemento obtiene el foco.

onreset: al resetear un formulario.

onselect: al seleccionar un elemento.

onsubmit: al enviar un formulario.

Eventos de imágenes:

onabort: se produce al interrumpirse la carga de una imagen.

EVENTOS

Eventos de teclado:

onkeydown: cuando se presiona una tecla.

onkeypress: al pulsar una tecla correspondiente a un carácter.

onkeyup: cuando se levanta una tecla tras pulsarla.

Eventos del ratón:

onclick: al hacer clic en un elemento.

ondblclick: cuando se hace doble clic en un elemento.

onmousedown: al pulsar un botón del ratón.

onmousemove: al mover el ratón dentro del área que ocupa el elemento.

onmouseout: al mover el ratón fuera del área del elemento.

onmouseover: al mover el ratón en el área de un elemento.

onmouseup: cuando se levanta el botón del ratón tras presionarlo.

EVENTOS

A continuación mostramos un ejemplo de cómo se definen eventos en HTML con la creación de funciones externas de Javascript:

```
<html>
    <head>
        <title>Página WEB de ejemplo: hiperenlaces</title>
        <script language="javaScript">
             function cargar() {
                 alert("La página ha sido cargada");
             }
             function pulsar() {
                 alert("Has escrito algo");
             }
             function procesar() {
                 alert("Haz hecho clic en el botón");
             }
        </script>
    </head>
     <body onload="cargar();">
         <input type="text" id="txt1" name="txt1" onkeypress="pulsar();" />
         <input type="button" id="btn1" name="btn1" value="Púlsame"
onclick="procesar();" />
    </body>
</html>
```

FIN